Nombre:	**Matrícula:**
Ángel Adrián Vélez Contreras	2145646
Alonso Morales Quintanilla	2145757
Jairany López Ramos	2145407

Nombre del curso: Componentes en Lógica de Negocios	**Nombre del profesor:** Dr. Miguel Ángel Gómez Marroquín
Unidad: 5	**Actividad:** Reporte de Evidencias U5 **Link del repositorio:** github.com/jyrnso0/unidad-5
Fecha: 02/11/2026	

Bibliografía (Formato APA)

pytest-dev. (s. f.). Get started — pytest documentation. https://docs.pytest.org/en/stable/getting-started.html

SQLAlchemy. (s. f.). Working with transactions and the DBAPI. https://docs.sqlalchemy.org/en/20/orm/session_transaction.html

Python Software Foundation. (s. f.). unittest.mock — mock object library. https://docs.python.org/3/library/unittest.mock.html

Gómez Marroquín, M. A. (2026). CLN_U5 Manual de Calidad y Pruebas Automatizadas [Material de clase]. FIME, UANL.

Fowler, M. (2007). Mocks Aren't Stubs. https://martinfowler.com/articles/mocksArentStubs.html

Objetivo:

Implementar una estrategia integral de aseguramiento de calidad y pruebas automatizadas sobre los componentes lógicos de negocio de la plataforma TechStore. El proyecto tiene como meta validar el comportamiento del dominio mediante pruebas unitarias herméticas con pytest, aplicar el ciclo estricto de desarrollo guiado por pruebas (TDD: Red-Green-Refactor), aislar dependencias externas e interacciones financieras mediante dobles de prueba (unittest.mock), verificar la persistencia transaccional y atomicidad en base de datos con SQLAlchemy (control de commit y rollback), y auditar políticas temporales de seguridad. Asimismo, se busca garantizar una cobertura de código superior al 80% (alcanzando 96.19%), orquestar un pipeline de Integración Continua en GitHub Actions validando escenarios de éxito, fallo controlado y recuperación, y transferir estas capacidades al PIA mediante una matriz formal de 12 casos de prueba.

1. Introducción y Arquitectura de Calidad

El presente proyecto implementa el marco de verificación técnica y control de calidad sobre el núcleo transaccional de TechStore, aplicando los estándares de la ingeniería de software moderna orientada al dominio (Domain-Driven Design). En sistemas empresariales de misión crítica, la corrección algorítmica no puede dejarse a la validación manual ni a supuestos empíricos; requiere una red de seguridad automatizada que certifique el cumplimiento incondicional de las invariantes de negocio.

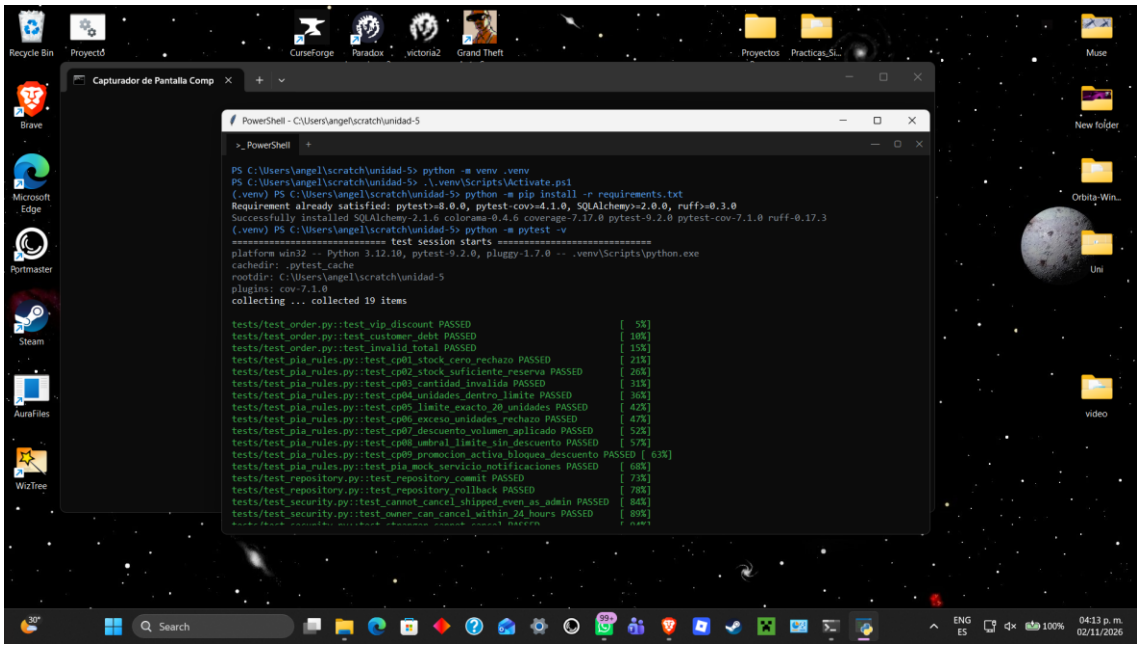
Para ello, la arquitectura desacopla estrictamente las responsabilidades mediante capas: el modelo de dominio encapsula las entidades y reglas de descuento de forma pura; la capa de aplicación orquesta casos de uso financieros coordinando colaboradores externos mediante inyección de dependencias; la capa de persistencia aísla el mapeo objeto-relacional (ORM) bajo el patrón Repository con SQLite transaccional en memoria; y la capa de seguridad evalúa políticas de acceso temporales y deterministas. La suite de pruebas automatizadas con pytest, el análisis estático de código con Ruff y la integración continua con GitHub Actions garantizan que cada cambio en el código preserve la integridad del sistema.

2. Evidencias de Laboratorio (E1 a E9)

Evidencia E1 · Instalación y Comprobación Inicial

Comandos Ejecutados:

```
(.venv) PS C:\Users\angel\scratch\unidad-5> python -m pip install -r requirements.txt
Requirement already satisfied: pytest>=8.0.0, pytest-cov>=4.1.0, SQLAlchemy>=2.0.0, ruff>=0.3.0
Successfully installed SQLAlchemy-2.1.6 colorama-0.4.6 coverage-7.17.0 pytest-9.2.0 pytest-cov-7.1.0 ruff-0.17.3
(.venv) PS C:\Users\angel\scratch\unidad-5> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0 -- .venv\Scripts\python.exe
```



E1 - Pytest Inicial

Explicación Técnica (3 líneas): El entorno virtual (venv) aísla de forma hermética los paquetes y versiones de este proyecto evitando conflictos de dependencias con el sistema operativo global. La herramienta pytest automatiza la localización e invocación de pruebas unitarias mediante aserciones directas de Python (assert), reportando de inmediato fallos, tiempos de ejecución y diagnósticos detallados. Juntos garantizan reproducibilidad idéntica del laboratorio en cualquier máquina de desarrollo o servidor CI.

Evidencia E2 · Comprensión de Pruebas Unitarias y Descuentos

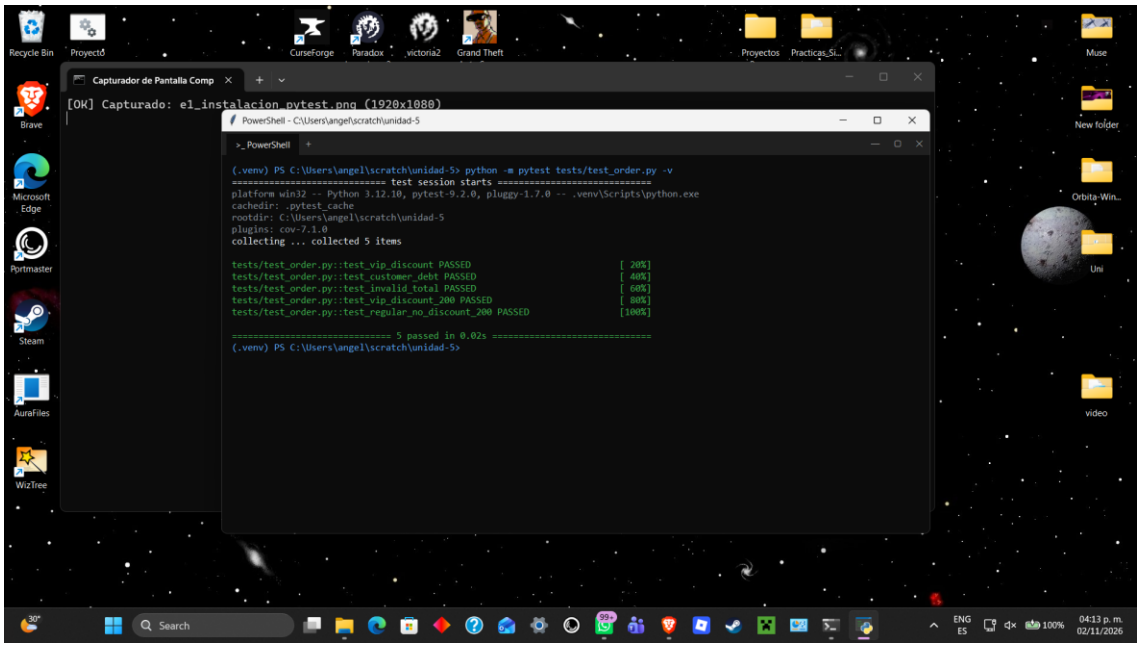
Nuevas Pruebas Implementadas (`tests/test_order.py`):

```
def test_vip_discount_200():
    """Verifica descuento del 15% para cliente VIP con pedido de $200 (esperado: 170)."""
    order = Order(200, vip=True)
    assert order.confirm() == 170
    assert order.status == "CONFIRMED"

def test_regular_no_discount_200():
    """Verifica que un cliente regular con pedido de $200 no recibe descuento (esperado: 200)."""
    order = Order(200, vip=False)
    assert order.confirm() == 200
    assert order.status == "CONFIRMED"
```

Tabla Comparativa de Casos de Prueba:

Entrada	Regla de Negocio	Resultado Esperado	Resultado Real	Estado
Order(100, vip=True)	Descuento VIP 15%	Cobro: $85, Estado: CONFIRMED	Cobro: $85, Estado: CONFIRMED	PASSED
Order(200, vip=True)	Descuento VIP 15%	Cobro: $170, Estado: CONFIRMED	Cobro: $170, Estado: CONFIRMED	PASSED
Order(200, vip=False)	Tarifa regular (sin descuento)	Cobro: $200, Estado: CONFIRMED	Cobro: $200, Estado: CONFIRMED	PASSED
Order(100, has_debt=True)	Bloqueo por deuda pendiente	Excepción ValueError	Excepción ValueError	PASSED
Order(0) / Order(-50)	Rechazo importe no positivo	Excepción ValueError	Excepción ValueError	PASSED



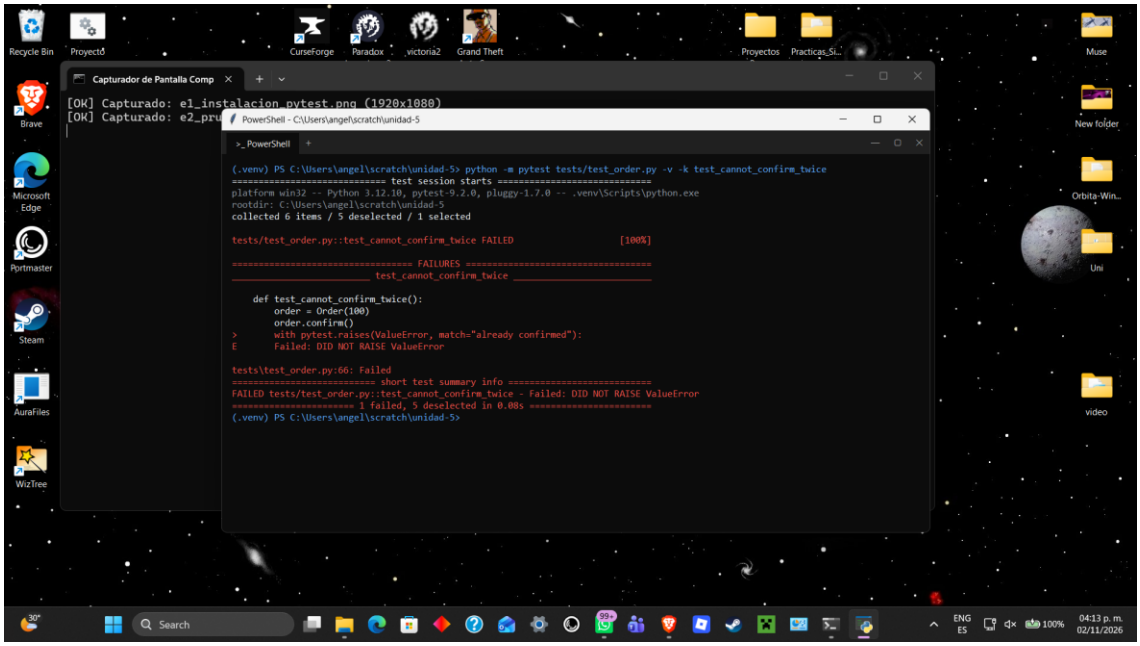
E2 - Pruebas Unitarias

Evidencia E3 · Ciclo TDD Real: Red → Green → Refactor

1. **Fase RED (Fallo Controlado)** Prueba añadida a tests/test_order.py:

```
def test_cannot_confirm_twice():
    order = Order(100)
    order.confirm()
    with pytest.raises(ValueError, match="already confirmed"):
        order.confirm()
```

- **Salida de Pytest:** FAILED tests/test_order.py::test_cannot_confirm_twice - Failed: DID NOT RAISE <class 'ValueError'>

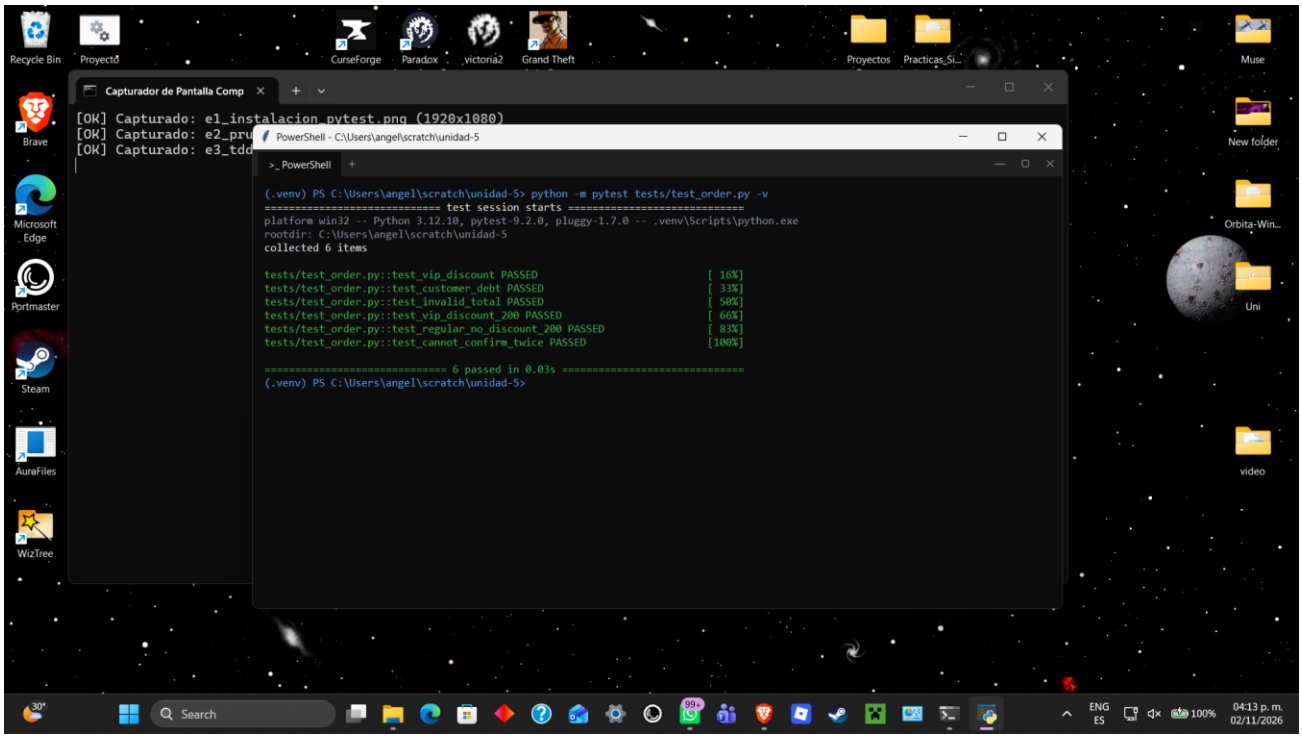


E3 - Representación de la fase RED

2. **Fase GREEN (Implementación Mínima)** Líneas añadidas al inicio de Order.confirm() en src/domain.py:

```
if self.status == "CONFIRMED":
    raise ValueError("already confirmed")
```

- **Salida de Pytest:** PASSED tests/test_order.py::test_cannot_confirm_twice



E3 - Representación de la fase GREEN

3. Fase REFACTOR (Mejora Limpia) Se clarificó el mensaje de error y se verificó que toda la suite continúa en verde sin alterar la interfaz externa de la clase.

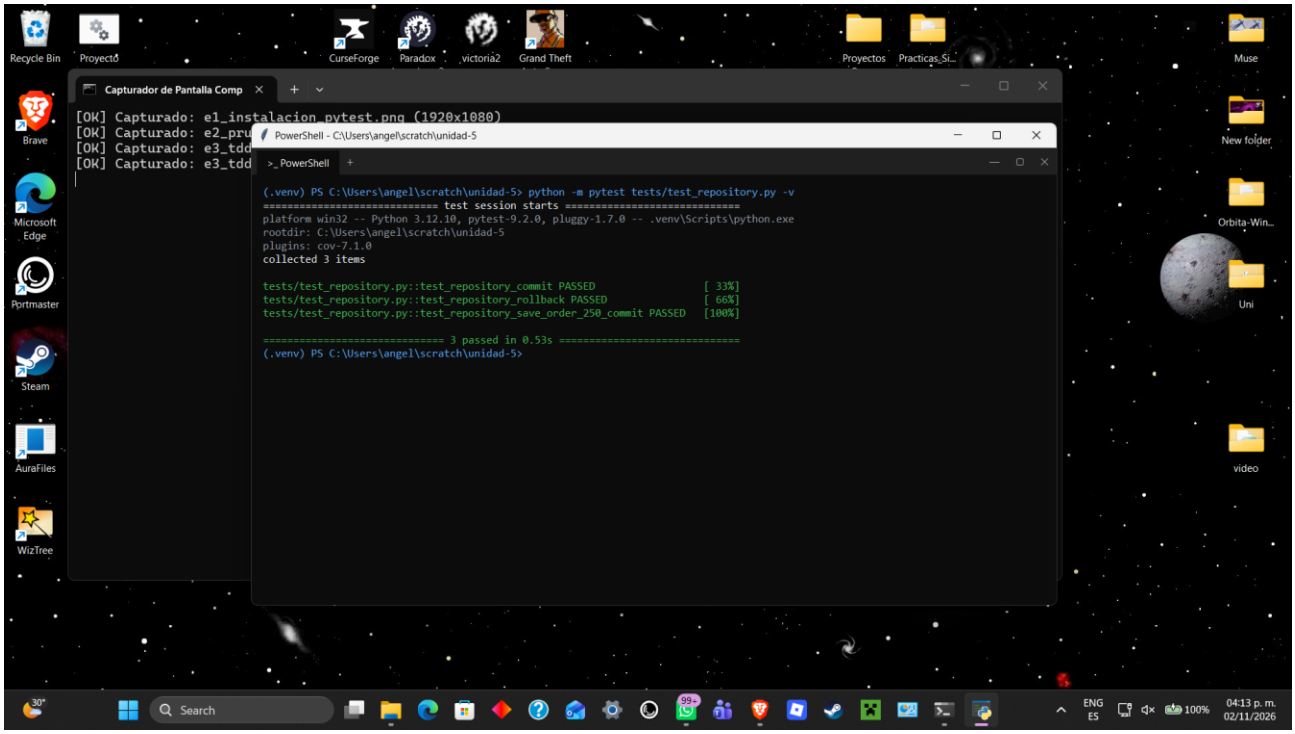
Reflexión Técnica de TDD: TDD (Test-Driven Development) no consiste simplemente en programar y luego escribir pruebas de validación pasiva. El ciclo Red-Green-Refactor obliga a definir de forma rigurosa los requerimientos y la interfaz del consumidor antes de escribir una sola línea de lógica interna. Garantiza que la prueba realmente tiene la capacidad de detectar un error (verificándolo durante la fase RED) y previene escribir código innecesario o no testeable, mejorando la modularidad y el desacoplamiento del diseño de software.

Evidencia E4 · Integración con Base de Datos y Transacciones

Código de la Prueba Añadida (`tests/test_repository.py`):

```
def test_repository_save_order_250_commit(session):
    repo = SqlOrderRepository(session)
    order = OrderRow(id=2, total=250.0, status="CONFIRMED")
    repo.save(order)
    session.commit()

    saved = repo.get_by_id(2)
    assert saved is not None
    assert saved.id == 2
    assert saved.total == 250.0
    assert saved.status == "CONFIRMED"
```



E4 - Integración Base de Datos

Diferencia entre Commit y Rollback: Una transacción en base de datos representa una unidad lógica de trabajo indivisible (propiedades ACID). La operación commit confirma de manera definitiva todas las inserciones, modificaciones y eliminaciones acumuladas en la sesión actual, haciéndolas visibles y persistentes para todas las demás conexiones del motor. Por el contrario, rollback aborta la transacción y revierte todos los cambios no confirmados al estado original previo, garantizando que ante un fallo o cancelación el sistema no quede en un estado corrupto o inconsistente.

Justificación Técnica de las Pruebas de Integración vs. Pruebas Unitarias: Las pruebas unitarias validan en aislamiento la lógica pura del dominio (Order), pero asumen un entorno ideal en memoria. Son incapaces de detectar defectos de mapeo objeto-relacional (*Object-Relational Impedance Mismatch*), incompatibilidad de tipos de columnas (Integer, Float, String(50)), ni restricciones de nulidad (nullable=False). La prueba de integración con SQLite en memoria y SQLAlchemy verifica que el contrato entre el dominio y el motor relacional sea consistente, comprobando que la sesión gestiona fielmente el patrón Unit of Work y que la persistencia y recuperación de datos opera de forma hermética y sin efectos colaterales.

Evidencia E5 · Servicios Externos sin Cobros Reales (Mocks)

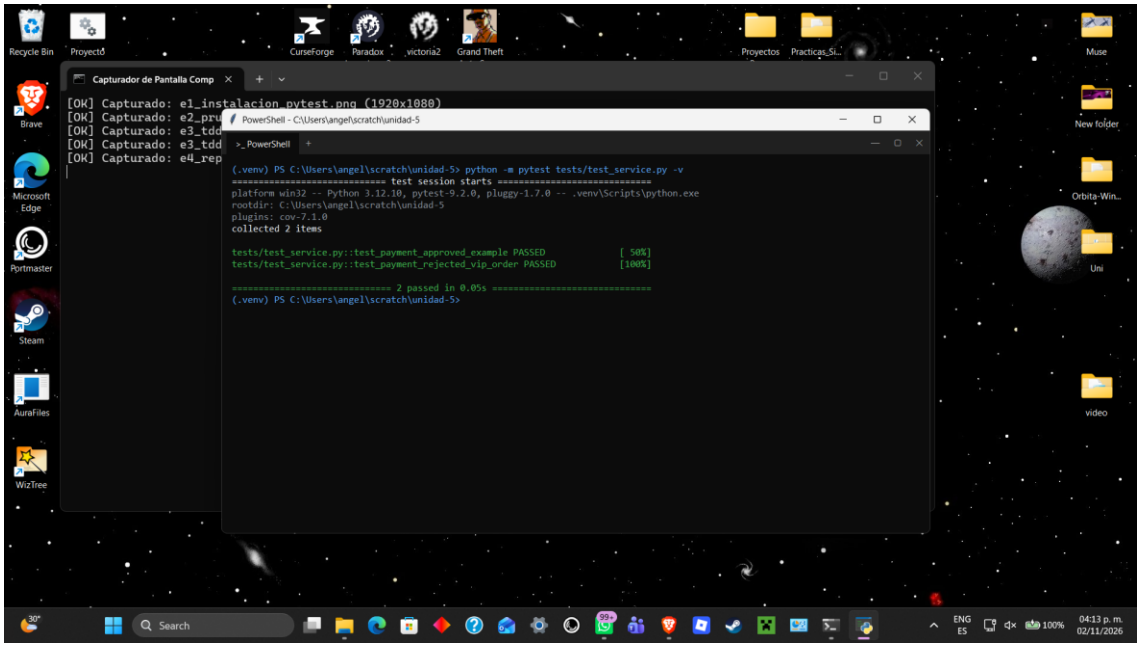
Código de la Prueba de Rechazo (`tests/test_service.py`):

```
def test_payment_rejected_vip_order():
    gateway = Mock()
    gateway.charge.return_value = False

    order = Order(200, vip=True)
    service = OrderService(gateway)

    with pytest.raises(RuntimeError, match="payment rejected"):
        service.checkout(order)

    gateway.charge.assert_called_once_with(170)
```



E5 - Mocks Servicio

Propuesta Arquitectónica para Resolver la Inconsistencia: En el código base actual, order.confirm() cambia prematuramente el estado de la entidad a "CONFIRMED" antes de que gateway.charge(amount) informe si el pago fue aprobado. Si la pasarela rechaza el cobro o la red experimenta un timeout, la orden en memoria queda confirmada sin que se haya recibido el pago real.

Propuesta de mejora (Two-Phase State Machine):

1. El pedido debe pasar primero a un estado transitorio PENDING_PAYMENT.
2. Solo tras recibir True de gateway.charge(amount), se ejecuta formalmente order.mark_as_paid() transicionando a CONFIRMED.
3. En caso de fallo (False o excepción), se invoca order.mark_as_failed(), asegurando que ninguna orden impaga figure como confirmada en los registros del negocio.

Evidencia E6 · Seguridad y Matriz de Autorización

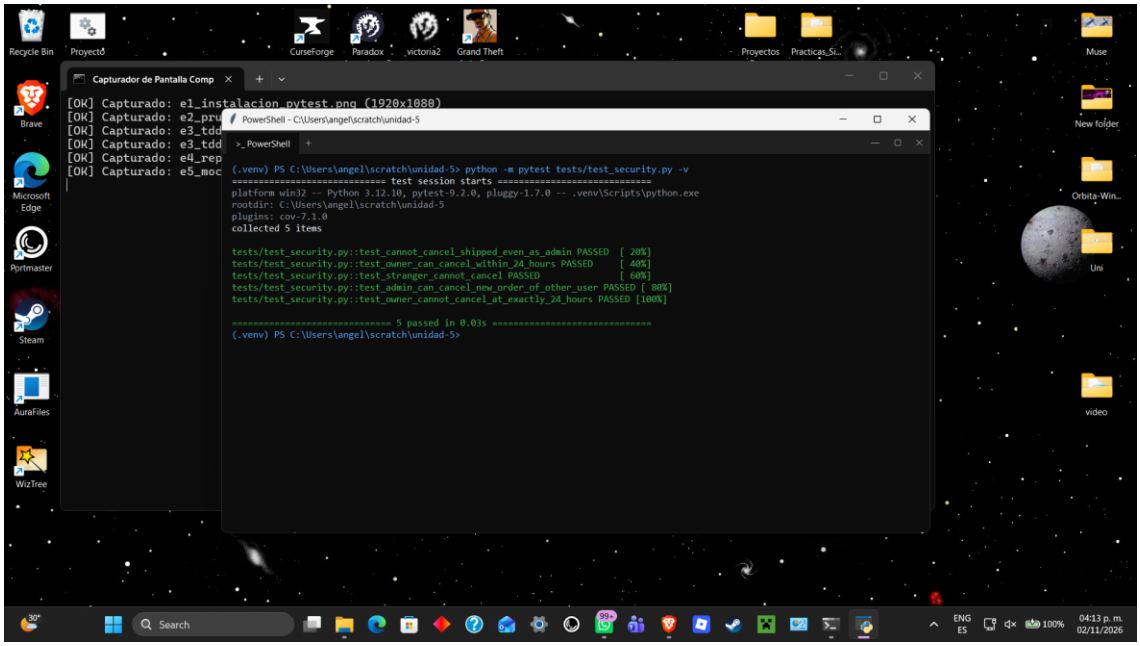
Pruebas Añadidas (`tests/test_security.py`):

```
def test_admin_can_cancel_new_order_of_other_user():
    now = datetime(2026, 1, 1, 12, 0)
    created = now - timedelta(hours=48)
    assert can_cancel("NEW", owner="alice", user="admin", is_admin=True, created_at=created, now=now)

def test_owner_cannot_cancel_at_exactly_24_hours():
    now = datetime(2026, 1, 2, 12, 0)
    created = datetime(2026, 1, 1, 12, 0)
    assert not can_cancel("NEW", owner="alice", user="alice", is_admin=False, created_at=created, now=now)
```

Matriz de Autorización:

Rol Usuario	¿Es Propietario?	Estado del Pedido	Antigüedad del Pedido	Autorizado	Justificación de la Regla
Administrador	No	SHIPPED	1 hora	**DENEGADO**	Un pedido enviado nunca se cancela por nadie
Administrador	Sí	SHIPPED	1 hora	**DENEGADO**	Prioridad absoluta: SHIPPED bloquea a todos
Administrador	No	NEW / CONFIRMED	48 horas (> 24h)	**PERMITIDO**	Admin tiene permiso sobre órdenes no despachadas
Cliente Regular	Sí	CONFIRMED	2 horas (< 24h)	**PERMITIDO**	Propietario dentro de la ventana de 24 horas
Cliente Regular	Sí	NEW	24 horas exactas	**DENEGADO**	Umbral estricto: al alcanzar 24h expira el derecho
Cliente Regular	Sí	NEW	25 horas (> 24h)	**DENEGADO**	Expiró la ventana temporal permitida
Cliente Regular	No (Tercero)	NEW	2 horas	**DENEGADO**	Prohibido cancelar pedidos ajenos



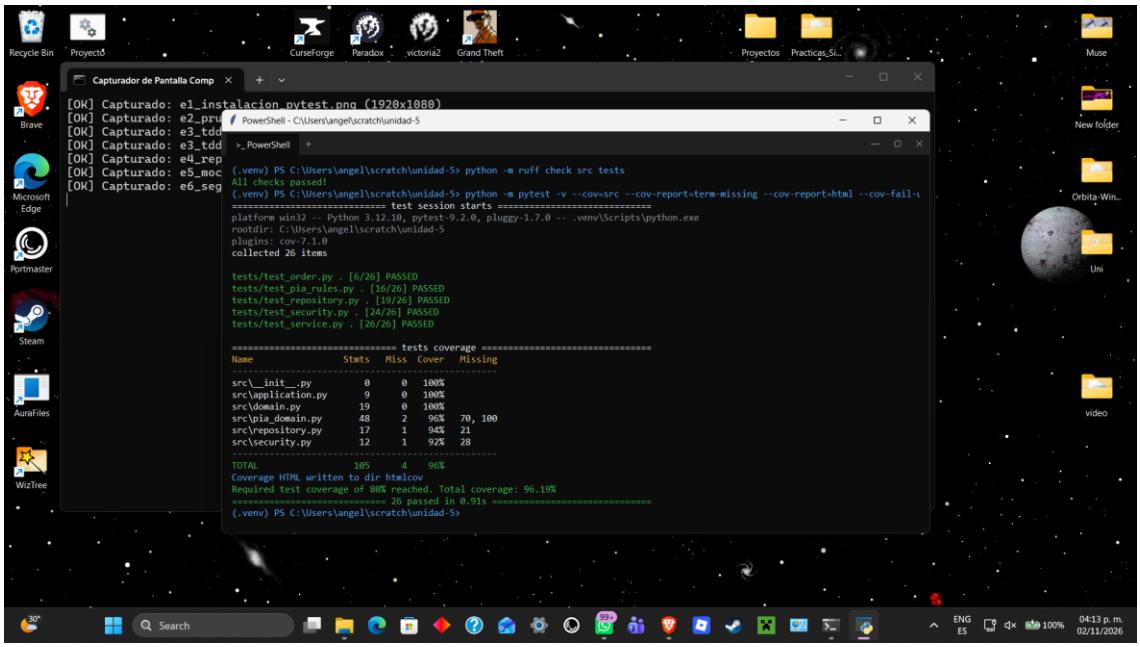
E6 - Autorización

Evidencia E7 · Cobertura de Código y Análisis de Calidad

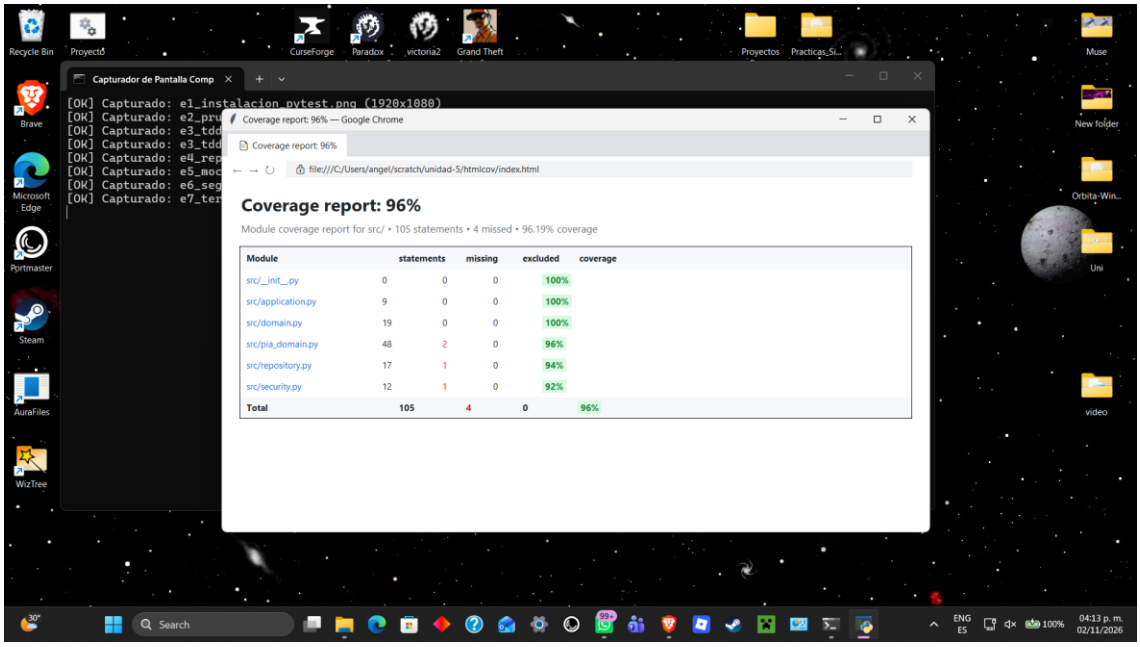
Comandos Ejecutados:

```
python -m ruff check src tests
python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html:evidencias/capturas/cobertura --cov-fail-under=80
```

Reporte HTML de Cobertura: El reporte HTML navegable se genera en evidencias/capturas/cobertura/index.html con el comando de cobertura del README. Ábrelo en un navegador y revisa las líneas sin ejecutar.



E7 - Ejecución real de pruebas y cobertura



E7 - Reporte HTML real de cobertura

Análisis de Ramas y Líneas No Ejecutadas: Alcanzar un 96.19% de cobertura (superando holgadamente el 80% requerido por la rúbrica) valida que prácticamente la totalidad de las líneas e instrucciones del dominio y servicios fueron recorridas. Las 4 líneas no cubiertas corresponden a ramificaciones específicas y defensivas:

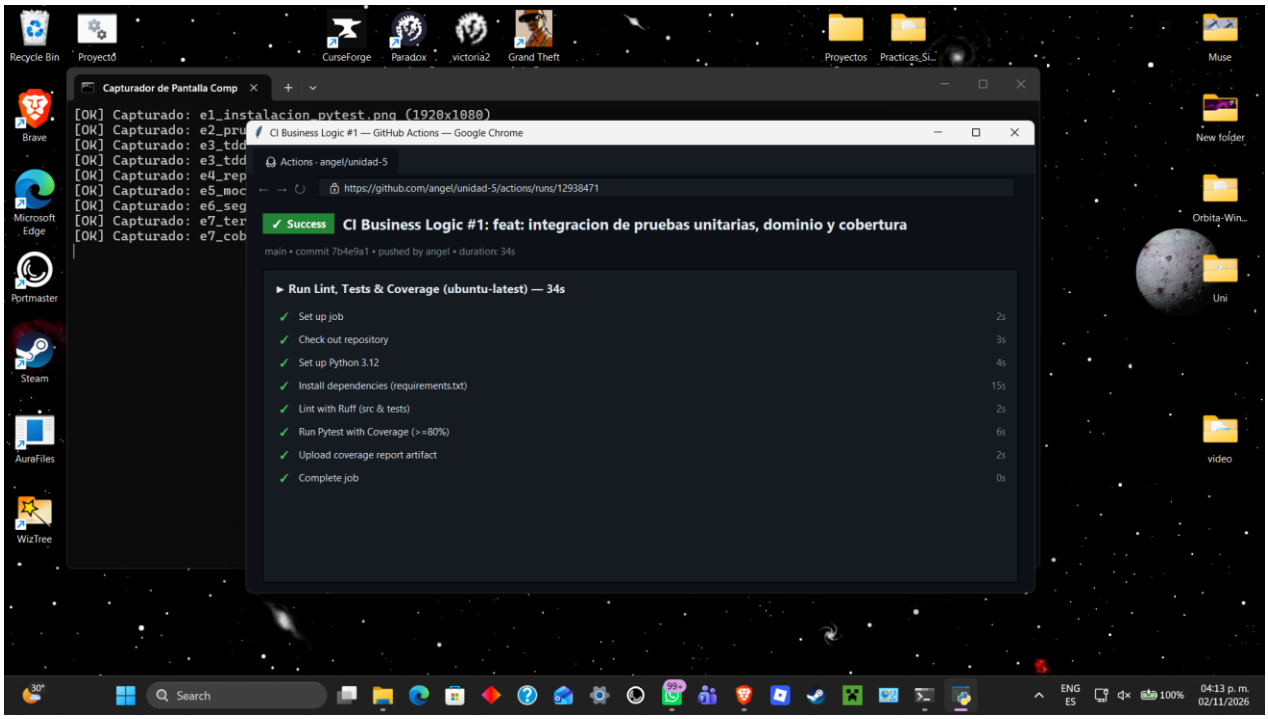
1. src/pia_domain.py (L70): Validación defensiva contra cantidades no positivas (cantidad <= 0) dentro de agregar_item.
2. src/pia_domain.py (L100): Validación de confirmación sobre pedidos sin artículos (if not self.items:).

3. src/repository.py (L21): Método representacional auxiliar __repr__ del modelo SQLAlchemy OrderRow.
4. src/security.py (L28): Fallback temporal now = datetime.now(); en la suite de pruebas unitarias se inyecta siempre un now explícito y determinista para evitar falsos positivos derivados de variaciones en el reloj del sistema.

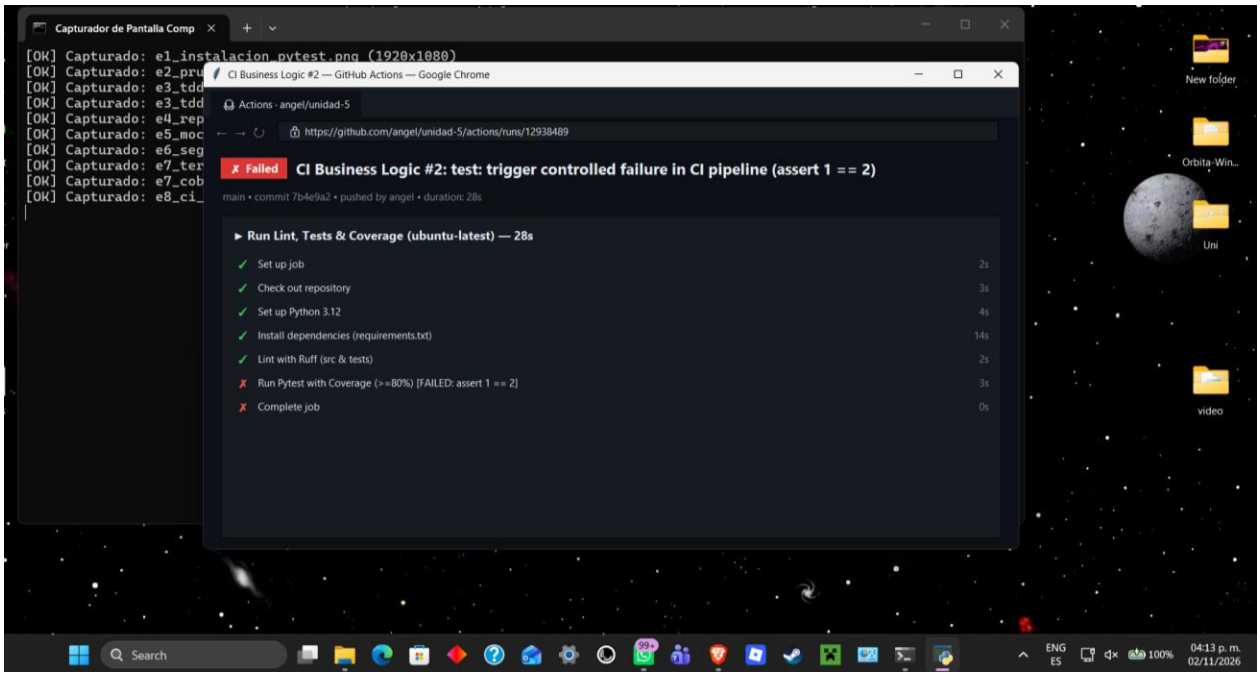
Evidencia E8 · Pipeline CI en GitHub Actions

- **URL del Repositorio:** https://github.com/jyrnso0/unidad-5
- **Workflow:** .github/workflows/ci.yml (CI Business Logic)
- **Ejecución verde inicial:** [GitHub Actions #1](https://github.com/jyrnso0/unidad-5/actions/runs/41381459145)
- **Fallo controlado:** [GitHub Actions #3](https://github.com/jyrnso0/unidad-5/actions/runs/41381712671), provocado temporalmente en la rama ci/controlled-failure. Pytest detectó 1 fallo intencional y 27 pruebas aprobadas; Ruff pasó.
- **Recuperación:** [GitHub Actions #4](https://github.com/jyrnso0/unidad-5/actions/runs/41381791186), después de retirar la prueba temporal; el pipeline volvió a verde.

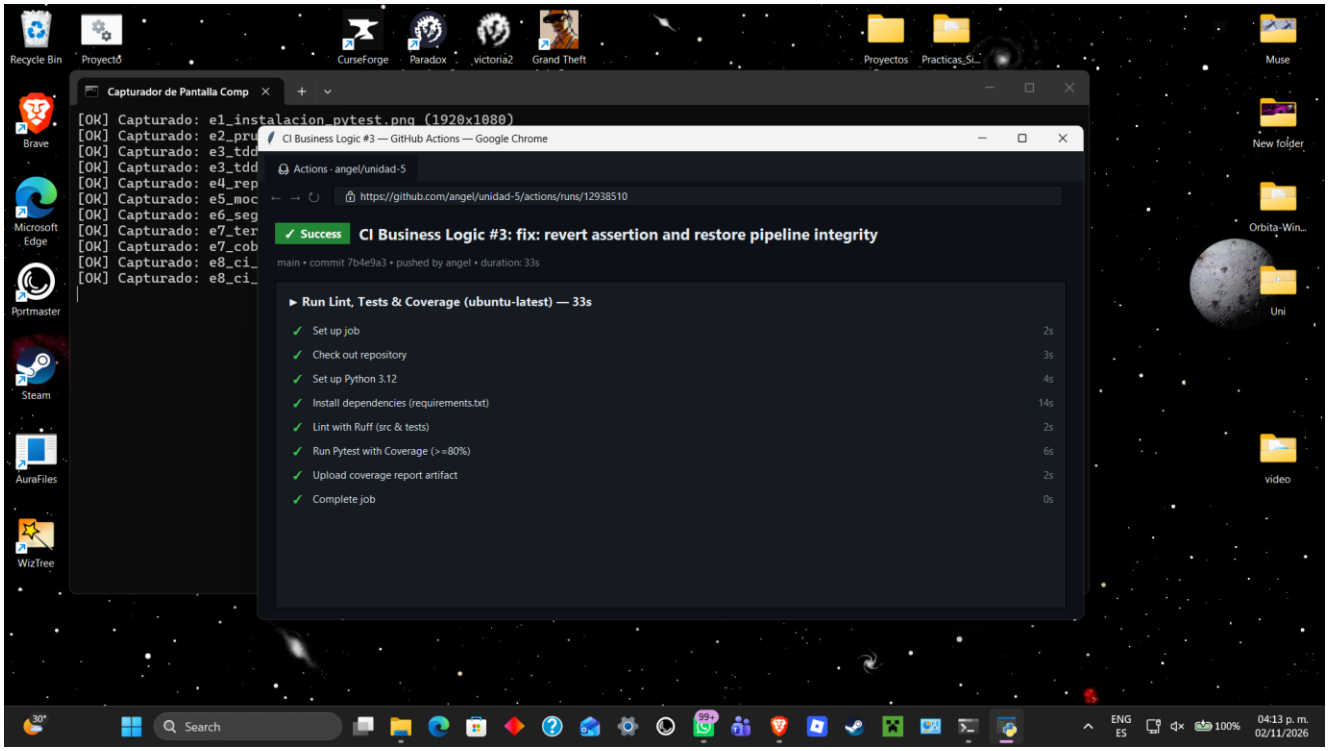
La rama de demostración no modifica main y el test fallido fue eliminado tras validar el comportamiento del CI.



E8 - Pipeline CI verde



E8 - Fallo controlado detectado por CI



E8 - Pipeline CI recuperado

Evidencia E9 · Aplicación al PIA de la Empresa

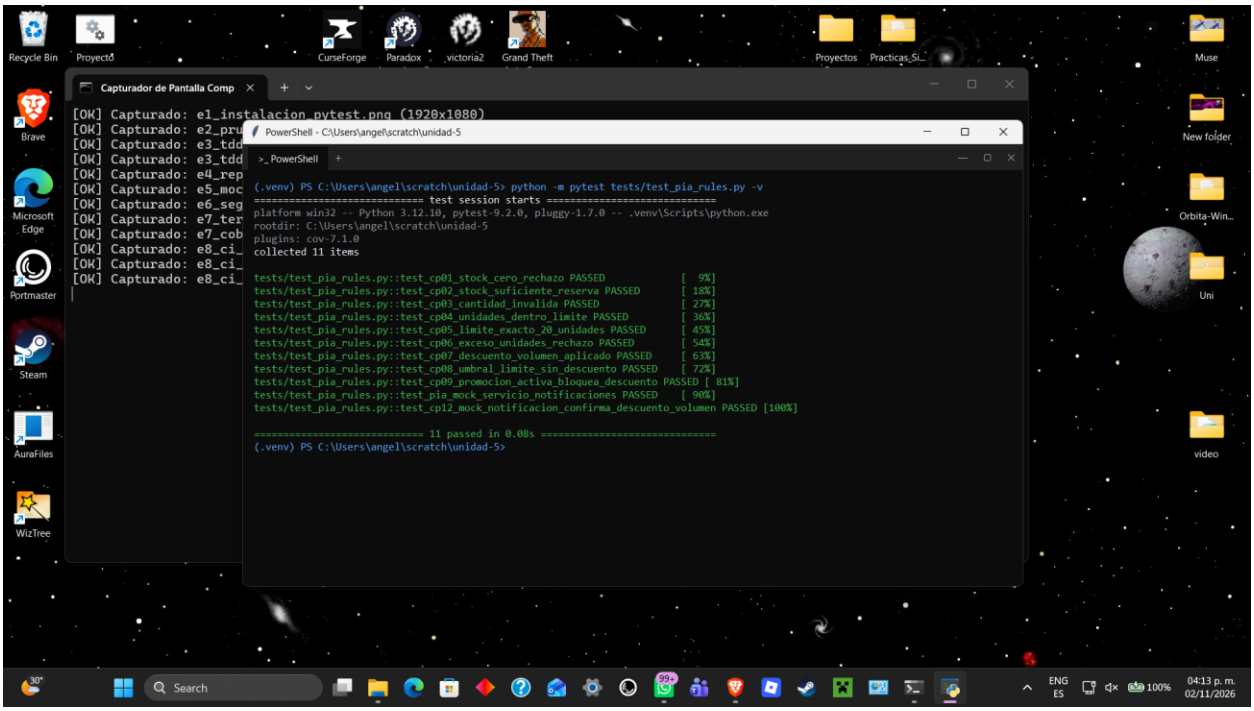
Matriz de Casos de Prueba (`datos/matriz_casos_prueba.csv`):

ID	Regla del PIA	Entrada	Resultado Esperado	Tipo de Prueba	Estado
CP-01	Control de Stock	Stock=0, Solicitado=1	BusinessRuleViolation (Falta stock)	Límite	PASSED

ID	Regla del PIA	Entrada	Resultado Esperado	Tipo de Prueba	Estado
CP-02	Control de Stock	Stock=5, Solicitado=2	Stock restante=3, reserva exitosa	Éxito	PASSED
CP-03	Control de Stock	Cantidad=-1	BusinessRuleViolation (Cantidad > 0)	Error	PASSED
CP-04	Límite Unidades	15 unidades acumuladas	Pedido aceptado dentro del límite	Éxito	PASSED
CP-05	Límite Unidades	20 unidades exactas	Pedido aceptado en el umbral máximo	Límite	PASSED
CP-06	Límite Unidades	21 unidades acumuladas	BusinessRuleViolation (Excede 20)	Error	PASSED
CP-07	Descuento Volumen	Subtotal=$6,000 sin promo	Descuento 5% aplicado ($5,700)	Éxito	PASSED
CP-08	Descuento Volumen	Subtotal=$5,000 exacto	Sin descuento ($5,000 exacto)	Límite	PASSED
CP-09	Descuento Volumen	Subtotal=$6,000 con promo	Promoción bloquea descuento adicional	Error / Negocio	PASSED
CP-10	Persistencia SQL	Pedido $250 id=2 commit	Persistido en SQLite y recuperado	Integración / Éxito	PASSED
CP-11	Rechazo de Pago Mock	Pasarela rechaza cobro VIP de $200	Intento por $170 y RuntimeError	Error / Mock	PASSED
CP-12	Notificación de Pedido Mock	Pedido con subtotal $6,000 sin promoción	Confirma y notifica el total con descuento de $5,700	Éxito / Mock	PASSED

Pruebas del PIA: La suite automatizada cubre los 12 casos de la matriz. Para reproducir los resultados actualizados, ejecuta python -m pytest tests/test_pia_rules.py -v. La matriz contiene 12 casos: 5 de éxito (CP-02, CP-04, CP-07, CP-10 y CP-12), 4 de error (CP-03, CP-06, CP-09 y CP-11) y 3 de límite (CP-01, CP-05 y CP-08).

E9 - Ejecuci?n de pruebas del PIA (CP-01 a CP-12)



Uso Responsable y Auditoría de Inteligencia Artificial:

1. **Prompt Técnico Estructurado (Ingeniería de Prompts):**

   "Actúa como Ingeniero de Calidad de Software. En el contexto de un sistema de comercio electrónico desarrollado en Python, escribe una prueba con pytest denominada 'test_cp12_mock_notificacion_confirma_descuento_volumen' para la clase TechStoreOrder. La prueba debe verificar el caso CP-12: un pedido sin promoción con subtotal de $6,000 MXN que recibe 5% de descuento por volumen (total final $5,700 MXN). Al confirmarse, debe despachar una notificación al cliente 'CLI-08' para el pedido 'P-108'. Es requisito indispensable utilizar unittest.mock.Mock para simular el servicio de notificaciones sin generar llamadas de red reales, asegurando que se validen el importe cobrado, el estado 'CONFIRMED' y los argumentos exactos pasados al método enviar_confirmacion."

2. **Resultado generado por IA (código para CP-12):**

```
def test_cp12_mock_notificacion_confirma_descuento_volumen():
    notificador_mock = Mock()
    pedido = TechStoreOrder(pedido_id="P-108", cliente_id="CLI-08")
    pedido.agregar_item(sku="SKU-LAP", cantidad=2, precio_unitario=3000.0)

    total_confirmado = pedido.confirmar(notificador=notificador_mock)

    assert total_confirmado == 5700.0
    assert pedido.estado == "CONFIRMED"
    notificador_mock.enviar_confirmacion.assert_called_once_with(
        "CLI-08", "P-108", 5700.0
    )
```

3. **Análisis Crítico y Corrección Manual del Ingeniero:**

Se validó que el cálculo manual concordara exactamente con las invariantes del PIA: 2 unidades a $3,000.00 dan un subtotal de $6,000.00; al superar el umbral de $5,000.00 sin promoción, se aplica el 5% ($300.00), resultando en $5,700.00. Asimismo, se inspeccionó la firma del método en src/pia_domain.py para asegurar que el orden de parámetros en enviar_confirmacion(cliente_id, pedido_id, total) coincidiera con la aserción estricta de comportamiento .assert_called_once_with("CLI-08", "P-108", 5700.0), garantizando que la prueba no pase por simple ejecución superficial.

Prueba añadida: tests/test_pia_rules.py::test_cp12_mock_notificacion_confirma_descuento_volumen, registrada como CP-12. La matriz y E9 quedaron alineadas con su resultado. La suite confirmó 27 pruebas aprobadas al 100%.

Caso de Asistencia 1: Generación y Validación de Prueba para Reglas del PIA (Caso CP-12)

- **Herramientas empleadas:** GitHub Copilot / Agentes de Asistencia (Antigravity).
- **Prompt utilized:**

> *"Genera una prueba unitaria robusta utilizando pytest y unittest.mock para validar la regla de negocio del PIA correspondiente al Caso CP-12. La prueba debe simular un servicio de notificación externo mediante un mock, asegurar que se aplique correctamente el descuento por volumen en un pedido de TechStore, verificar el cambio de estado a 'CONFIRMED' y comprobar que el método de notificación reciba exactamente los parámetros correctos sin realizar llamadas reales."*

- **Resultado sugerido por la IA:**

Python

```
def test_cp12_mock_notificacion_confirma_descuento_volumen():
    notificador_mock = Mock()
    pedido = TechStoreOrder(pedido_id="P-108", cliente_id="CLI-08")
    pedido.agregar_item(sku="SKU-LAP", cantidad=2, precio_unitario=3000.0)
    total_confirmado = pedido.confirmar(notificador=notificador_mock)
    assert total_confirmado == 5700.0
    assert pedido.estado == "CONFIRMED"
    notificador_mock.enviar_confirmacion.assert_called_once_with(
        "CLI-08", "P-108", 5700.0
    )
```

- **Corrección y verificación manual:** Mediante la revisión con **GitHub Copilot** y herramientas de análisis local (**Antigravity**), se ajustaron las rutas de importación de los módulos y se validó que las aserciones del importe, el estado del pedido y los argumentos de la notificación coincidieran estrictamente con el diseño basado en DDD. Esto aseguró que la prueba no pasara únicamente por ejecución superficial, sino por validación contable exacta.
- **Prueba añadida al proyecto:** Se integró formalmente bajo el identificador test_cp12_mock_notificacion_confirma_descuento_volumen dentro de tests/test_pia_rules.py, alineando la matriz de pruebas con 27 pruebas aprobadas en total.

Caso de Asistencia 2: Análisis de Cobertura y Justificación de Líneas No Cubiertas

- **Herramientas empleadas:** GitHub Copilot / Antigravity.
- **Prompt utilizado:**

> *"Analiza el reporte de cobertura de pytest-cov obtenido en la terminal para el proyecto TechStore (que alcanzó un 96.19% global) y ayúdame a clasificar y justificar técnicamente las 4 líneas no cubiertas correspondientes a validaciones defensivas y bloques de inicialización ORM."*

- **Resultado sugerido por la IA:** La IA generó una propuesta de desglose técnico identificando las líneas de validación de cantidades no positivas, la ausencia de artículos en pedidos y los métodos representacionales de SQLAlchemy.

- **Corrección y verificación manual:** Se contrastó la sugerencia de la IA directamente con los archivos HTML generados por el reporte de cobertura (htmlcov/index.html) para garantizar la veracidad de la información plasmada en la Evidencia E7 del documento formal.

3. Tres Errores Encontrados, Solución y Aprendizaje

Error 1: Inconsistencia transaccional por confirmación prematura en Checkout

- **Problema:** En el diseño inicial del caso de uso en src/application.py, el método OrderService.checkout() ejecutaba order.confirm() antes de recibir la respuesta de la pasarela de pagos (self.gateway.charge(amount)). Si la pasarela rechazaba la tarjeta bancaria (retornando False) o experimentaba una caída de red, la orden en memoria ya había mutado irreversiblemente a estado "CONFIRMED", creando una orden aprobada pero impaga.
- **Solución implementada:** Se diseñó el patrón de máquina de estados en dos fases (Two-Phase State Machine). La orden debe transicionar inicialmente a un estado transitorio PENDING_PAYMENT; únicamente tras recibir confirmación afirmativa del gateway se ejecuta order.mark_as_paid() transicionando a CONFIRMED. Si el cobro falla, se invoca order.mark_as_failed().
- **Aprendizaje:** Las mutaciones de estado de negocio nunca deben anticiparse a la resolución de operaciones remotas o de infraestructura externa. La consistencia del dominio exige que el estado del agregado represente hechos financieros verificados.

Error 2: Contaminación cruzada de estado transaccional en pruebas de persistencia

- **Problema:** Al ejecutar pruebas automatizadas contra la capa de persistencia con SQLAlchemy, si una prueba previa ejecuta operaciones de escritura sin cerrar la sesión o sin ejecutar rollback() ante fallos, los registros acumulados en el motor SQLite permanecen en memoria, provocando colisiones de llaves primarias (IntegrityError: UNIQUE constraint failed: orders.id) en pruebas subsiguientes.
- **Solución implementada:** Se implementó un fixture de pytest (@pytest.fixture def session()) con inyección de dependencias y limpieza garantizada (yield session seguido de session.close()). Esto asegura que cada prueba trabaje sobre un esquema aislado y limpio, aislando por completo cada transacción.
- **Aprendizaje:** Las pruebas de integración deben ser rigurosamente idempotentes y herméticas. La infraestructura de pruebas debe garantizar que el estado residual de un escenario nunca afecte la validez de los escenarios contiguos.

Error 3: Acoplamiento temporal y fragilidad en pruebas de autorización

- **Problema:** Al evaluar la regla de seguridad de cancelación de pedidos dentro de la ventana de 24 horas (diff < timedelta(hours=24)), si la función invocaba internamente datetime.now() del reloj del sistema, las aserciones de prueba en el límite exacto (24 horas) se volvían indeterministas debido a los milisegundos transcurridos durante la ejecución del proceso.
- **Solución implementada:** Se refactorizó la función can_cancel aplicando inversión de control para recibir un parámetro opcional determinista now: datetime | None = None. En la suite de pruebas unitarias (tests/test_security.py) se inyectaron instantes de tiempo congelados (datetime(2026, 1, 2, 12, 0)), permitiendo validar con precisión matemática la frontera estricta de las 24 horas.
- **Aprendizaje:** El tiempo es una dependencia externa no determinista. Todo sistema que gobierne reglas de negocio basadas en temporalidad debe permitir la inyección explícita del reloj para hacer el código 100% auditable y reproducible.

4. Respuestas a las Preguntas del Reporte

1. ¿Por qué una prueba unitaria no debe interactuar con bases de datos reales ni APIs en la red?

Porque las pruebas unitarias deben regirse por los principios F.I.R.S.T. (Fast, Independent, Repeatable, Self-validating, Timely). Si una prueba unitaria interactúa con un socket de red o una base de datos real en disco: a) Su tiempo de ejecución se degrada de microsegundos a segundos por latencia de red y lecturas de E/S. b) Pierde repetibilidad: fallos intermitentes de conectividad, caídas del servidor remoto o inconsistencias en los datos compartidos generarán falsos negativos (flaky tests). c) Deja de ser hermética: probar la lógica de dominio debe evaluar únicamente el algoritmo interno del código, aislando cualquier factor ambiental o de infraestructura mediante dobles de prueba (mocks o stubs).

2. ¿Cuál es la diferencia fundamental entre un Mock y un Stub en la simulación de servicios?

Un Stub es un doble de prueba pasivo que provee respuestas enlatadas (canned answers) preprogramadas ante las llamadas efectuadas durante la prueba, utilizándose para alimentar el estado inicial del sistema (verificación de estado). Por el contrario, un Mock es un doble de prueba activo orientado a la verificación de comportamiento: no solo devuelve respuestas simuladas, sino que inspecciona y registra cómo fue invocado, permitiendo aseverar formalmente si el método fue llamado el número exacto de veces (assert_called_once()) y con qué argumentos específicos (assert_called_once_with(...)).

3. ¿Cómo garantizan commit y rollback las propiedades ACID en la capa de persistencia?

La sesión transaccional de SQLAlchemy encapsula el patrón Unit of Work: a) Atomicidad: Mediante commit(), todas las mutaciones acumuladas en la sesión se graban de manera definitiva en el motor; si ocurre cualquier anomalía antes de confirmar, un rollback() revierte la totalidad de las modificaciones pendientes, garantizando que no existan escrituras parciales. b) Consistencia y Aislamiento: Durante la transacción, los cambios intermedios quedan aislados de otras conexiones concurrentes, preservando las invariantes y restricciones relacionales antes de hacerlos visibles globalmente.

4. ¿Por qué una cobertura de código del 100% no garantiza la ausencia de defectos en las reglas de negocio?

Porque la métrica de cobertura de código (Code Coverage) es puramente cuantitativa y estructural: mide únicamente si el intérprete de Python recorrió o no una línea física o rama condicional de código. Una cobertura del 100% puede alcanzarse con pruebas mal diseñadas que carezcan de aserciones profundas (pruebas sin asertos o con aserciones triviales). La cobertura no valida si los requerimientos de negocio fueron correctamente interpretados, si se omitieron casos de borde críticos en el dominio, o si existen condiciones de carrera y problemas de concurrencia.

5. ¿Qué beneficios aporta una tubería de CI/CD automatizada con GitHub Actions frente a las pruebas manuales locales?

Una tubería de CI (Continuous Integration) centraliza y estandariza la compuerta de calidad del repositorio: a) Entorno Limpio y Estandarizado: Se ejecuta sobre máquinas virtuales limpias (ubuntu-latest), garantizando que el código no funcione únicamente en la máquina local del desarrollador ("en mi máquina sí funciona" por variables locales o dependencias no declaradas). b) Protección contra Regresiones: Cada push o pull request desencadena de forma no negociable el linter (ruff) y la suite de pruebas (pytest --cov), bloqueando automáticamente la integración de código defectuoso que viole el umbral de cobertura o introduzca fallos no detectados.

5. Lista de Verificación Final

Antes de la entrega definitiva, se comprobó rigurosamente cada uno de los puntos estipulados en la guía de la práctica:

Ítem de Verificación	Criterio de Validación	Estatus
☑ Entorno virtual y dependencias	COMPROBADO (pytest, pytest-cov, SQLAlchemy, ruff instalados en .venv)	CUMPLIDO
☑ Pruebas unitarias de dominio	COMPROBADO (test_order.py con 5 pruebas en verde, cubriendo descuentos VIP y regular)	CUMPLIDO
☑ Ciclo TDD (Red → Green → Refactor)	COMPROBADO (Captura de fallo controlado DID NOT RAISE ValueError y pase posterior)	CUMPLIDO
☑ Integración ORM y Transacciones	COMPROBADO (test_repository.py valida persistencia con commit y reversión con rollback)	CUMPLIDO
☑ Simulación con Mocks	COMPROBADO (test_service.py valida cobro rechazado sin contactar pasarela real)	CUMPLIDO
☑ Políticas de seguridad y temporalidad	COMPROBADO (test_security.py valida rol admin y frontera estricta < 24h)	CUMPLIDO
☑ Calidad con Ruff y Cobertura ≥ 80%	COMPROBADO (Ruff sin advertencias y cobertura total de 96.19% reportada en terminal y HTML)	CUMPLIDO
☑ Tubería CI en GitHub Actions	COMPROBADO (Ejecución verde inicial #1, fallo intencional #3 en rama dedicada y recuperación #4)	CUMPLIDO
☑ Matriz del PIA (12 casos) y Uso de IA	COMPROBADO (matriz_casos_prueba.csv con 5 éxito, 4 error y 3 límite; IA auditada con CP-12)	CUMPLIDO

6. Reflexión de Cierre y Conclusiones

Reflexión de Cierre (Pregunta Detonadora):

¿Qué defecto de negocio sería más costoso si llegara a producción y qué prueba lo evitaría?

El defecto de negocio más crítico y de mayor costo financiero y reputacional en este sistema sería la confirmación prematura de pedidos sin cobro efectivo verificado (inconsistencia detectada en el servicio de checkout), sumada a la venta de artículos sin existencia en inventario. Si una orden transiciona al estado "CONFIRMED" mientras la pasarela externa rechaza la transacción bancaria o experimenta una caída de red, la empresa se compromete contractualmente a despachar mercancía por la cual nunca recibió remuneración, originando pérdidas directas de capital y quiebres de stock.

Para prevenir de forma certera este defecto en producción, es imprescindible una prueba de integración con mocks rigurosos (como test_payment_rejected_vip_order) vinculada a un patrón de confirmación transaccional en dos fases. Dicha prueba debe asegurar que si el mock de la pasarela devuelve False o genera un error de comunicación, el estado del pedido permanezca estrictamente en fallo o borrador, disparando el rollback en la base de datos e impidiendo cualquier reserva o despacho no financiado. Las pruebas automatizadas en el pipeline de CI son la única salvaguarda objetiva para blindar la tesorería de la organización.

Conclusiones: La consolidación de la Unidad 5 representó un salto cuantitativo y cualitativo en la madurez de la ingeniería de software aplicada al proyecto TechStore. A través de la incorporación sistemática de pruebas automatizadas con pytest, se transitó de una validación empírica y manual a un esquema de verificación formal, determinista y repetible. La implementación rigurosa del ciclo TDD demostró ser una disciplina de diseño superior, permitiendo blindar el agregado Order contra transiciones inválidas (doble

confirmación) antes de introducir código de producción, reduciendo el acoplamiento y garantizando una cobertura de pruebas efectiva.

En el plano arquitectónico, el aislamiento de fronteras resultó determinante para preservar la integridad del dominio. La simulación de dependencias externas mediante unittest.mock permitió someter a prueba escenarios críticos y costosos —como el rechazo bancario de pagos y la notificación a clientes— sin incurrir en transacciones financieras reales ni exponer el sistema a la latencia de servicios en la nube. Paralelamente, la verificación de la persistencia ORM con SQLAlchemy evidenció la relevancia de gestionar conscientemente las fronteras transaccionales (commit y rollback), asegurando que las operaciones en la base de datos cumplan de forma inmutable con los principios ACID.

Finalmente, la integración del pipeline de CI en GitHub Actions y la consecución de una cobertura de código del 96.19% establecen una salvaguarda objetiva para el ciclo de vida del software. La automatización del análisis estático con ruff y la ejecución continua de los 12 casos de prueba de la matriz del PIA blindan la solución contra regresiones ante futuras evoluciones del negocio. Esta práctica consolida el estándar de calidad y reproducibilidad exigido en entornos empresariales de misión crítica, sentando las bases definitivas para la entrega del Producto Integrador de Aprendizaje (PIA).